Redis Radar

In Redis Software

Private Preview Guide

Version: v0.3.0
Deployment model: Customer-managed, on-premises
Audience: Approved Private Preview participants

1. Overview

Redis Radar is a centralized control plane that provides a single pane of glass for managing Redis Enterprise fleets across customer-controlled environments.

Redis Radar helps operations, platform, and infrastructure teams:

- Understand fleet-wide health across connected Redis Enterprise clusters
- Identify cluster, node, and database issues faster
- Gain basic capacity awareness
- Navigate from fleet-level visibility into cluster, node, and database-level details
- Improve operational visibility across large Redis environments

Redis Radar is intended to complement existing monitoring and observability tools. It is not a replacement for real-time monitoring, alerting, or enterprise observability platforms.

---

2. Private Preview Scope

Redis Radar v0.3.0 is an early Private Preview release intended for:

- Design partner validation
- Proof-of-Concept deployments
- Evaluation of core workflows
- Feedback on product direction, usability, and operational fit

> ⚠ **Important**
> Redis Radar v0.3.0 is not intended for production use.
> It should be deployed only in controlled, non-production environments.
> Features, APIs, and UX may change without prior notice.
> No guaranteed upgrade path to future releases.
> Limited security validation compared with a GA release.

---

3. Terms and Conditions

By clicking "Accept" or by accessing, downloading, installing, or using this Preview in any way, you acknowledge that you have read, understood, and agree to be bound by the following terms.

This Preview is provided under and governed by Section 3.2 (Previews) of the Redis Software Agreement (the "Agreement"). By accepting these terms, you confirm that you have the authority to bind your organization to the Agreement.

Specifically, by installing, accessing, or using Redis Radar v0.3.0, you acknowledge and agree that:

1. Redis Radar v0.3.0 is provided as a Private Preview release.
2. Use of this Private Preview is subject to the Redis Enterprise Software Agreement, including Section 3.2, Previews.
3. Redis Radar v0.3.0 is not generally available and is not intended for production use.
4. Redis may modify, discontinue, or substantially change functionality, behavior, APIs, installation methods, or user experience in future versions.
5. Redis does not guarantee compatibility or upgradeability from this Private Preview version to future versions.
6. Participation in this Private Preview requires active collaboration with Redis, including providing feedback within two weeks of installation.

---

4. Key Capabilities in v0.3.0

Fleet Visibility

Redis Radar provides a unified view of connected Redis Enterprise clusters, including fleet-level health status, healthy/warning/critical indicators, aggregated risk signals, cluster connectivity and last-seen status, and entry points for drill-down investigation.

Cluster and Topology Exploration

Users can navigate from the fleet view into specific clusters to inspect cluster metadata, health, nodes, databases, connectivity status, and relevant health indicators.

Basic Capacity Awareness

Redis Radar provides high-level capacity indicators to help identify high-utilization clusters, potential capacity pressure points, and nodes or clusters that may require attention. This version does not provide advanced capacity planning, forecasting, or real-time capacity analytics.

Active-Active and Replication Awareness

- Active-Active database indication
- Replication status at the database level
- Fleet-level aggregated replication view

5. Compatibility & Authentication

Requirement	Details
Minimum Redis Enterprise version	Redis Enterprise Software 8.0.16 or later
Required APIs	health_report, usage_report, crdbs, actions
Authentication (supported)	Local authentication, LDAP, SAML
Data freshness — health	Approximately every 5 minutes
Data freshness — usage	Snapshot-based, approximately hourly
Max clusters (Private Preview)	Approximately 100 Redis Enterprise clusters

6. Kubernetes & OpenShift Deployment

Redis Radar provides a Helm chart for Kubernetes-based environments. This option is intended for evaluation, platform integration testing, OpenShift validation, and customer environments that prefer Kubernetes-native deployment patterns.

Architecture

The Kubernetes deployment mirrors the standard Radar architecture:

Redis Radar Component	Kubernetes Resource
Application (mcm-api)	Deployment
Async Worker (mcm-worker)	Deployment
Migration (mcm-migrate)	Job (runs once per install/upgrade)
PostgreSQL	External (recommended); optional bundled Bitnami subchart for non-production

Prerequisites

Before deploying, confirm:

- Kubernetes 1.23+ (chart uses autoscaling/v2 and batch/v1 Job TTL)
- Helm 3.x installed

- Validated on OpenShift 4.x
- An external PostgreSQL instance (recommended for production; bundled Bitnami subchart available for non-production only)
- Outbound network access from the cluster to your Redis Enterprise clusters and image registry
- Credentials with appropriate permissions for each target Redis Enterprise cluster
- Your organization has approved the use of this Private Preview in a non-production environment

Step 1 — Create required Secrets

Before running helm install, create two Kubernetes Secrets out-of-band.

PostgreSQL connection string

```
kubectl create secret generic radar-db \
  --namespace radar \

--from-literal=DATABASE_URL='postgres://radar:secret@postgres.example.com:5432/
radar?sslmode=require'
```

Credential encryption key (required for LDAP; 32 bytes)

```
ENC_KEY=$(openssl rand -hex 16)
kubectl create secret generic radar-credentials \
  --namespace radar \
  --from-literal=CREDENTIAL_ENCRYPTION_KEY="$ENC_KEY" \
  --from-literal=CREDENTIAL_ENCRYPTION_KEY_VERSION=v1
```

Step 2 — Install the Helm chart

Standard Kubernetes install (with Ingress):

```
helm install radar ./helm/radar \
  --namespace radar \
  --create-namespace \
  --set database.existingSecret=radar-db \
  --set credentials.existingSecret=radar-credentials \
  --set ingress.enabled=true \
  --set ingress.className=nginx \
  --set ingress.hosts[0].host=radar.example.com \
  --set ingress.hosts[0].paths[0].path=/ \
  --set ingress.hosts[0].paths[0].pathType=Prefix
```

For OpenShift, use the provided values file instead:

```
helm install radar ./helm/radar \
  --namespace radar \
  --create-namespace \
```

```
  -f ./helm/radar/values-openshift.yaml \
  --set database.existingSecret=radar-db \
  --set credentials.existingSecret=radar-credentials \
  --set route.host=radar.apps.example.com
```

Step 3 — Verify the installation

7. Check all pods are running:

```
kubectl get pods -n radar
# Expect: radar-mcm-api-*, radar-mcm-worker-*, radar-mcm-migrate-* (Completed)
```

8. Confirm the migration job completed:

```
kubectl logs -n radar job/radar-mcm-migrate
```

9. Check health endpoints:

```
kubectl port-forward -n radar svc/radar 8080:80
curl http://localhost:8080/healthz/ready   # 200 OK
curl http://localhost:8080/healthz/live    # 200 OK
```

10. Run the chart-provided readiness test:

```
helm test radar --namespace radar
```

11. Access the Redis Radar UI using the configured Ingress hostname or OpenShift Route.
12. Create the initial admin user on first login.
13. Add your first Redis Enterprise cluster connection.
14. Verify connectivity using the cluster "Last Seen" status.

> ⚠ **Important**
> The Kubernetes deployment is available but not production-certified in this Private Preview.
> The following areas are still evolving: private registry authentication, production secrets management,
> upgrade and rollback workflows, disaster recovery, high availability, and customer-specific TLS/DNS/route policies.
> The chart does not require anyuid, privileged SCCs, host paths, or cluster-admin permissions.

7. Getting Started

After installation, complete the following quick start steps:

15. Deploy Redis Radar using the Kubernetes Helm chart.
16. Create the initial admin user.
17. Access the Redis Radar UI.
18. Add a connection to a Redis Enterprise cluster.

19. Verify connectivity using the "Last Seen" status.
20. Explore the fleet overview.
21. Drill down into cluster, node, and database details.
22. Document feedback and share it with Redis within two weeks of installation.

8. Support During Private Preview

Redis Radar v0.3.0 is supported through an engineering-led Private Preview process. Support includes:

- Installation assistance
- Troubleshooting support
- Issue triage
- Feedback sessions
- Product and workflow discussions

This Private Preview does not include the same support model, SLA, or escalation process as a generally available Redis product. Please reach out to your Redis contact for any questions.

9. Known Limitations

Redis Radar v0.3.0 has the following known limitations:

- Not intended for production use
- On-premises, single-instance deployment only
- No high availability or backup and restore
- No guaranteed upgrade path
- No cloud integrations
- No license management or bulk operations
- No advanced capacity planning or real-time monitoring
- Kubernetes deployment is not production-certified
- Features, APIs, and UX may change rapidly